Reinforcement Learning for PI Controller Tuning

1.0 Learning Outcomes.....1
2.0 Introduction.....1
3.0 Reinforcement Learning Fundamentals.....2
4.0 Actor-Critic Learning and TD3.....4
5.0 Designing an RL Tuner for a PI Controller.....4
6.0 Example: RL Tuning of a Virtual DC Motor (Simulation).....6
7.0 Limitations and Trust.....8
8.0 Summary.....8
References.....9
Recommended Citation.....9

1.0 Learning Outcomes

After studying this ACE-Lab learning guide, you should be able to demonstrate the Learning Outcomes listed in Table 1. On completion of the ACE-Lab learning guide, revisit Table 1 and confirm whether each Learning Outcome has been achieved.

Table 1: *Learning Outcomes from the Reinforcement Learning for PI Controller Tuning Learning Guide*

ID	Learning Outcome	Achieved
1.	Define the main elements of reinforcement learning (RL): agent, environment, observation, action, reward, policy and return.	☐
2.	Explain how an actor-critic method such as TD3 learns a continuous control policy from experience.	☐
3.	Design the observations, actions and reward for an RL agent that tunes the gains of a PI controller.	☐
4.	Interpret the results of an RL tuner against the engineering requirements.	☐
5.	Evaluate the limitations of learning-based control, including transparency, generalisation and the evidence needed before it can be trusted.	☐

2.0 Introduction

The Adaptive Control: the Self-Tuning PI Controller Learning Guide showed how a controller can re-tune itself by identifying a model of the system online and redesigning its gains from the model. Every step was based on an explicit model and an explicit design rule. A different approach is to let the controller learn how to tune itself from experience, by trial and error, without being told the model or the design rule. An important question therefore arises:

Can a controller learn a good tuning strategy from experience, and how far can such a strategy be trusted?

This is the idea behind reinforcement learning (RL), a branch of machine learning in which an agent learns to choose actions that maximise a reward. RL has achieved notable results in games and robotics, and it is increasingly promoted for control engineering. In this Learning Guide, RL is used in a deliberately limited and transparent way: the agent does not replace the PI controller, but supervises it, choosing the proportional and integral gains every few seconds. The PI controller remains in the loop, so the familiar feedback structure, its actuator limits and its anti-windup are kept.

Why is Reinforcement Learning Important?

RL is important in control engineering because:

i. It can learn a control or tuning strategy for systems that are difficult to model, or where the best strategy is not obvious.

ii. Once trained, the policy is a small function that is quick to evaluate and can run on a microcontroller.

iii. It provides a way to optimise directly for the performance that matters, such as tracking error, overshoot and actuator activity, through the reward.

iv. Engineers increasingly need to judge claims made for artificial intelligence (AI) in control, which requires an understanding of how these methods work and where they can fail.

3.0 Reinforcement Learning Fundamentals

The Agent and the Environment

RL describes a decision-making problem as an interaction between an agent and an environment, as shown in Figure 1. At each decision step t, the agent receives an observation, o_t, of the environment, and chooses an action, a_t. The environment responds: its state changes, and it returns a new observation and a scalar reward, R_{t+1}, that measures how good the result was. The rule that maps observations to actions is called the policy, $a_t = \pi(o_t)$. The aim of learning is to find the policy that maximises the expected return, the discounted sum of future rewards:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{i=0}^{\infty} \gamma^i R_{t+i+1} \tag{1}$$

where γ ($0 \le \gamma < 1$) is the discount factor. A small γ makes the agent focus on immediate rewards, and a value close to 1 makes it plan further ahead. The interaction is divided into episodes, each of which starts from a new initial condition, and the agent improves its policy from the experience collected over many episodes.

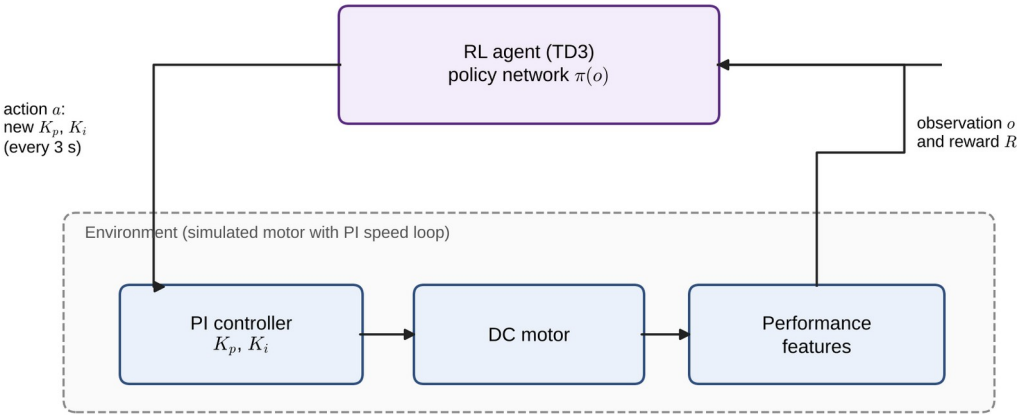


Figure 1: *The RL Agent Supervises the PI Speed Loop: Every 3 s it Observes the Performance and Chooses New Gains*

Table 2 relates the general terms of RL to the PI tuner developed in this Learning Guide.

Table 2: *Reinforcement Learning Terms Applied to the PI Tuner*

RL term	Meaning	In the ACE-Lab PI tuner
Agent	The learner and decision maker	A TD3 agent with a small neural network policy
Environment	Everything the agent interacts with	The PI speed loop and the DC motor (simulated during training)
Observation, o_t	What the agent measures	Ten numbers describing recent tracking performance and the motor response
Action, a_t	What the agent decides	The PI gains, K_p and K_i
Reward, R_{t+1}	How good the outcome was	Penalises tracking error, overshoot and chatter
Decision interval	Time between actions	3 s, one reference step
Episode	One training run	90 s of operation (30 decisions) on a randomised motor

Value Functions and Exploration

To improve its policy, the agent must estimate how good each action is. The action-value function, $Q^{\pi}(o,a)$, is the expected return when action a is taken after observation o and the policy π is followed afterwards. It satisfies the Bellman equation, which links the value now to the reward and the value one step later:

$$Q^{\pi}(o_t, a_t) = \mathbb{E}\left[R_{t+1} + \gamma\, Q^{\pi}\left(o_{t+1}, \pi(o_{t+1})\right)\right] \tag{2}$$

A policy that always chooses the action with the highest estimated value can only improve on what it already knows. The agent must also explore: during training, random noise is added to its actions so that it tries new gains and discovers whether they are better. The amount of exploration is reduced as training progresses. This balance between exploration and exploitation is central to RL (Sutton and Barto, 2018).

4.0 Actor-Critic Learning and TD3

The gains of a PI controller are continuous quantities, so the agent must choose actions from a continuous range rather than from a short list. Actor-critic methods are well suited to this. They use two function approximators, usually neural networks:

- The **actor** is the policy, $a = \pi_\phi(o)$, with parameters ϕ: it maps an observation directly to an action.
- The **critic**, $Q_w(o,a)$, with parameters w, estimates the value of taking action a after observation o.

The critic is trained to satisfy the Bellman equation, Equation (2), using stored experience $(o_t, a_t, R_{t+1}, o_{t+1})$, and the actor is trained to choose actions that the critic rates highly. The Twin Delayed Deep Deterministic policy gradient (TD3) algorithm (Fujimoto et al., 2018) adds three refinements that make this process more reliable:

i. **Twin critics:** two critics are trained, and the smaller of their two estimates is used to calculate the training target, which reduces the tendency of value estimates to be over-optimistic:

$$y_t = R_{t+1} + \gamma \min_{i=1,2} Q_{w_i'}\left(o_{t+1},\, \pi_{\phi'}(o_{t+1}) + \epsilon\right) \tag{3}$$

ii. **Target policy smoothing:** a small, clipped random noise, ϵ, is added to the target action in Equation (3), so that the value estimate does not exploit narrow peaks in the critic.

iii. **Delayed policy updates:** the actor, and the slowly changing target networks marked with a prime, are updated less often than the critics, so that the policy is improved using a more settled estimate of value.

During training, the critics are adjusted to reduce $(y_t - Q_{wi}(o_t,a_t))^2$, and the actor is adjusted to increase $Q_{w1}(o_t,\pi_\phi(o_t))$. After training, only the actor is needed to run the controller.

5.0 Designing an RL Tuner for a PI Controller

The success of RL depends less on the learning algorithm than on how the problem is posed: what the agent can observe, what it is allowed to do, what it is rewarded for, and what it experiences during training. This section describes each choice for the ACE-Lab tuner.

Decision Interval and Structure

The PI controller runs every T_s = 0.01 s, using the velocity form so that the output does not jump when the gains change. The agent acts much less often, every T_a = 3 s, which is one step of the square-wave speed reference. Each decision therefore sees the complete response to one reference step, which provides a meaningful measure of performance, and the agent cannot destabilise the loop by changing the gains too quickly.

Observations

At the end of each interval, the agent receives ten observations, scaled to be of the order of one:

$$o_t = \left[\frac{e_{RMS}}{10},\ \frac{OS}{10},\ \frac{\bar{u}}{V_s},\ \frac{\bar{y}}{50},\ \frac{\hat{K}}{30},\ \frac{\hat{K}_{prev}}{30},\ \frac{e_{RMS,prev}}{10},\ \frac{OS_{prev}}{10},\ a_1,\ a_2 \right]^T \tag{4}$$

where e_{RMS} is the RMS speed error over the interval, OS the overshoot past the new reference (RPM), $\bar{u}$ and $\bar{y}$ the mean voltage and speed, and a_1, a_2 the current action. The term $\hat{K}$ is the change in mean speed divided by the change in mean voltage between successive intervals, an incremental gain that is unaffected by the dead-band. It gives the agent a simple measure of how strongly the motor responds, so that it can detect when the motor changes. The previous values allow the agent to see trends.

Actions

The actor outputs two numbers, a_1 and a_2, between −1 and 1 (a tanh output). These are mapped to the gains on a logarithmic scale, so that the agent can choose small and large gains with equal resolution:

$$K_p = 10^{-2.5+1.4(a_1+1)}, \qquad K_i = 10^{-1.3+1.1(a_2+1)} \tag{5}$$

giving $0.003 \le K_p \le 2$ V/RPM and $0.05 \le K_i \le 8$ V/(RPM s). The ranges themselves are a safety limit: the agent cannot choose gains outside them.

Reward

The reward is calculated at the end of each interval and expresses the requirements as a single number to be maximised:

$$R = -\left(\frac{e_{RMS}}{5}\right)^2 - \frac{OS}{2} - 0.15\,c \tag{6}$$

where c is the chatter, the mean absolute change in the PWM command per sample (in counts). The first term rewards accurate tracking, the second penalises overshoot, and the third discourages high gains that amplify encoder noise into the motor voltage. The weights express the relative importance of these goals, and changing them changes the behaviour that is learned. The agent will exploit any weakness in the reward: if chatter were not penalised, for example, it would learn to use very high gains.

Training on Randomised Motors

If the agent were trained on one motor, it would learn the gains for that motor only. Instead, each training episode uses a different virtual motor, with a random gain, time constant and dead-band, and a random load change (the motor gain is multiplied by a factor between 0.6 and 2) at a random time part-way through. This is known as domain randomisation. To earn a high reward in every episode, the agent must learn to recognise how the motor is behaving, from its observations, and to choose suitable gains: the same job as the self-tuning PI controller, but learned rather than designed. The training settings are summarised in Table 3.

Table 3: *Training Settings for the TD3 PI Tuner (`train_rl_tuner.m`)*

Setting	Value
Actor (policy) network	10–16–16–2, ReLU hidden layers, tanh output (482 parameters)
Critic networks	Two critics, 64 hidden units
Discount factor, γ	0.7
Mini-batch size, experience buffer	128, 10^5

Setting	Value
Exploration noise	Standard deviation 0.4, decaying to 0.05
Episodes	1000 episodes of 90 s (30 decisions each)
Training time	About 25 minutes in parallel on a 16-core PC

The actor, shown in Figure 2, is deliberately small. With 482 parameters, under 2 KB in single precision, the trained policy can be exported as plain MATLAB code (`export_rl_policy.m`) and run on the Arduino Mega 2560 without the Reinforcement Learning Toolbox. The discount factor of 0.7 makes the agent focus on the next few reference steps, which suits a tuner that must respond to the current motor.

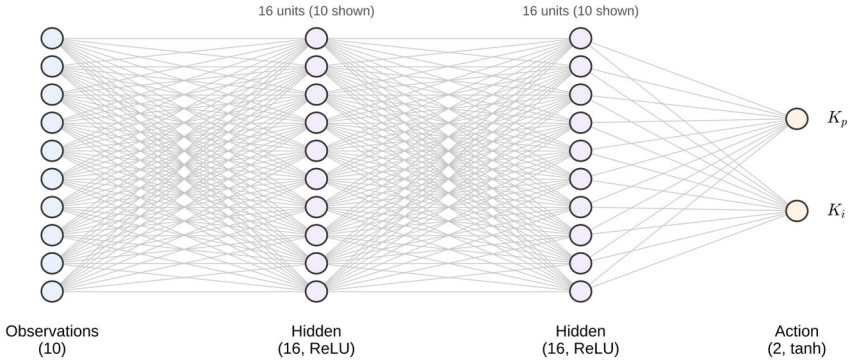


Figure 2: *The Actor (Policy) Network: Ten Observations are Mapped to the Two PI Gains*

MATLAB Script: creating and training the TD3 agent (extract from `train_rl_tuner.m`)

```
mdl = 'rl_pi_tuner';  Ta = 3;  T_ep = 90;
obsInfo = rlNumericSpec([10 1]);                          % observations
actInfo = rlNumericSpec([2 1], 'LowerLimit', -1, 'UpperLimit', 1);
env = rlSimulinkEnv(mdl, [mdl '/RL Agent'], obsInfo, actInfo);
env.ResetFcn = @(in) randomMotor(in, mdl);                % new motor

initOpts  = rlAgentInitializationOptions('NumHiddenUnit', 64);
agentOpts = rlTD3AgentOptions('SampleTime', Ta, 'DiscountFactor', 0.7, ...
    'MiniBatchSize', 128, 'ExperienceBufferLength', 1e5);
agentOpts.ExplorationModel.StandardDeviation = 0.4;
agentOpts.ExplorationModel.StandardDeviationDecayRate = 1e-4;
agentOpts.ExplorationModel.StandardDeviationMin = 0.05;
agent = rlTD3Agent(obsInfo, actInfo, initOpts, agentOpts);

anet = dlnetwork([featureInputLayer(10)                   % small actor
                  fullyConnectedLayer(16); reluLayer
                  fullyConnectedLayer(16); reluLayer
                  fullyConnectedLayer(2);  tanhLayer]);
agent = setActor(agent, rlContinuousDeterministicActor(anet, ...
                obsInfo, actInfo));

trainOpts = rlTrainingOptions('MaxEpisodes', 1000, ...
    'MaxStepsPerEpisode', ceil(T_ep/Ta), 'Plots', 'training-progress');
stats = train(agent, env, trainOpts);
```

6.0 Example: RL Tuning of a Virtual DC Motor (Simulation)

In this example, the trained agent tunes the PI speed controller of a virtual motor (τ = 0.25 s, 0.5 V dead-band) whose gain changes part-way through the run. In scenario A the motor starts with K = 16 RPM/V and its gain falls to 60% at t = 40

s, as if a load were added; in scenario B it starts with K = 10 RPM/V and its gain doubles, as if a load were removed. The reference alternates between 20 and 40 RPM every 3 s. The responses are shown in Figure 3, and the results are summarised in Table 4, using the engineering requirements of the earlier practical exercises: a rise time below 1 s (R1), a steady-state error of at most 0.1 RPM (R2), an overshoot of at most 5% (R3) and chatter of at most 5 PWM counts per sample (R4), with R5 requiring R1 to R4 to be met again for every step from 50 s.

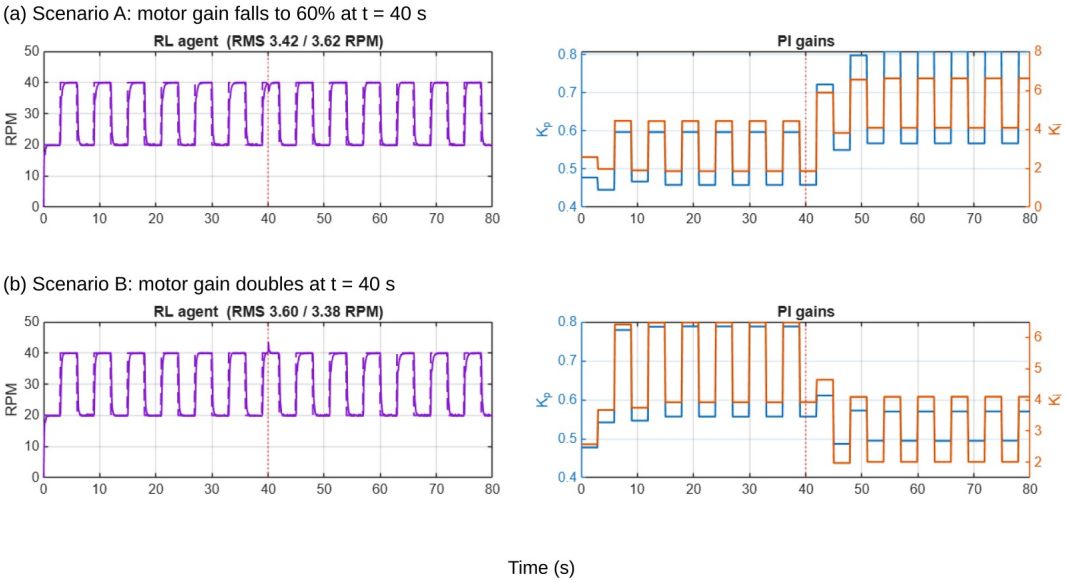


Figure 3: *Speed (Left) and PI Gains Chosen by the RL Tuner (Right) for Scenario A (a) and Scenario B (b); the Gain of the Motor Changes at t = 40 s*

Table 4: *Simulation Results for the RL Tuner (Worst Case over the Steps in Each Window)*

Scenario	RMS error, 20–40 s (RPM)	RMS error, 60–80 s (RPM)	Worst overshoot, 10–40 s (%)	Worst overshoot, 50–80 s (%)	K_p, K_i at the end	R5 met
A	3.42	3.62	1.6	1.6	0.808, 6.616	Yes
B	3.60	3.38	17.6	1.6	0.570, 4.097	Yes

Several observations can be drawn from these results:

- **The agent re-tuned quickly.** After each change, it settled on new gains within one or two 3 s decisions, and from 50 s it met every requirement in both scenarios, with a worst-case overshoot of 1.6%.
- **The gains moved in the expected direction.** When the motor weakened (scenario A), the agent raised both gains; when the motor became stronger (scenario B), it lowered them. It did this without an explicit model, using only its observations of how the speed responded to the voltage.
- **The agent learned to treat upward and downward steps differently.** The gains alternate at every reference step, because the agent can see the mean speed and voltage, and has learned that different gains suit the two directions of step. This behaviour was not designed: it emerged from the reward.
- **The behaviour before the change was not perfect.** In scenario B, the agent allowed overshoots of up to 17.6% before the change, which fails R3, even though

it met every requirement after the change. A single reward that balances several goals does not guarantee that each requirement is met.

- **The reasons for the decisions are not visible.** The gains can be observed, but the reason for choosing K_p = 0.81 rather than any other value is contained only in the 482 parameters of the network.

7.0 Limitations and Trust

The results show that a learned tuner can perform very well, but they also illustrate why learning-based control must be treated with care:

i. **Transparency:** the policy is a neural network, and its reasoning cannot be read from its weights. It is difficult to explain, or to certify, why a particular pair of gains was chosen.

ii. **Generalisation:** the agent has no basis for its decisions outside the range of motors and conditions it experienced in training. A motor with a different time constant, a sensor fault, or a disturbance never seen in simulation may produce unexpected gains. The policy used in the simulation example must be retrained around the identified hardware motor before it is used on the ACE-Lab.

iii. **The simulation-to-reality gap:** training relies on a simulator. Effects that are missing from the simulator, such as encoder quantisation, friction that changes with temperature, or delays in the hardware, are not learned.

iv. **Reward design:** the agent optimises exactly what the reward measures, which may not be exactly what the engineer intended.

v. **Cost:** RL needs a representative simulator, a carefully designed reward and around 25 minutes of training on a powerful computer.

Before a learned controller could be trusted in a real product, evidence would be needed of its performance on the real hardware, across the full range of expected operating conditions, together with safety limits that do not depend on the policy, such as the gain ranges and actuator limits used here, and monitoring that can revert to a conventional controller if its behaviour becomes unexpected.

8.0 Summary

This Learning Guide introduced reinforcement learning (RL), in which an agent learns a policy that maximises a discounted sum of rewards through interaction with an environment. Actor-critic methods learn a policy (the actor) and an estimate of its value (the critic), and the TD3 algorithm makes this reliable using twin critics, target policy smoothing and delayed policy updates. An RL tuner was designed for the ACE-Lab PI speed controller: every 3 s, the agent observes the tracking performance and the response of the motor, and chooses new gains on a logarithmic scale, with a reward that penalises tracking error, overshoot and chatter. Training on randomised virtual motors teaches the agent to re-tune when the motor changes. The key progression introduced in this guide can be summarised as:

simulator + reward → training by trial and error → policy → gains every 3 s → evidence on hardware before trust

In simulation, the RL tuner re-tuned within one or two decisions after the motor changed and then met every requirement, but its decisions cannot be explained

and its performance outside its training conditions cannot be guaranteed. The tuner is explored on the ACE-Lab in the practical exercise, DC Motor PI Tuning using Reinforcement Learning.

References

Fujimoto, S., van Hoof, H. and Meger, D. (2018). Addressing function approximation error in actor-critic methods. *Proceedings of the 35th International Conference on Machine Learning*, PMLR 80, pp. 1587–1596.

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D. and Wierstra, D. (2016). Continuous control with deep reinforcement learning. *International Conference on Learning Representations (ICLR)*.

Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd edn. Cambridge, MA: MIT Press.

Recommended Citation

If you have used this learning guide in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). Reinforcement Learning for PI Controller Tuning. ACE-Lab. Teaching Notes: ACE-Lab Learning Guide, Version 1.0. Zenodo. https://doi.org/10.5281/zenodo.23671547